PROJECT SENTINEL

Complete Context Document

National Fraud Interdiction Grid

Full-Stack AI-Powered Fraud Detection System

Python (FastAPI) + React (Vite) + Google Gemini AI

Last Updated: July 12, 2026

This document contains everything about the Project Sentinel codebase - architecture, file structure, full source code, API contracts, setup instructions, and developer notes. Any AI assistant or developer reading this should be able to immediately understand, run, and modify this project.

TABLE OF CONTENTS

TABLE OF CONTENTS

1. PROJECT OVERVIEW

Project Sentinel is a prototype National Fraud Interdiction System for India. It is a full-stack web application that uses Google Gemini AI to provide:

1. Live Call Interception (Vishing Detection): A real-time microphone listener that transcribes phone calls using the browser's Web Speech API, sends the transcript to a Python backend, which then uses Google Gemini AI to detect if the call is a scam (Digital Arrest, Customs Fraud, OTP Fraud, Money Mule).
2. Counterfeit Currency Scanner (Computer Vision): An image upload feature where users can upload photos of Indian Rupee banknotes. The image is sent to the Gemini AI Vision model, which performs a deep forensic scan for missing RBI security features (microprinting, security thread, intaglio printing, serial number anomalies, watermarks).
3. AI Fraud Assistant (Chat): A WhatsApp-style conversational chatbot where citizens can paste suspicious SMS messages, payment links, or describe incidents. The AI analyzes the text and returns a threat score.
4. Nodal Officer Dashboard: A government-style command center UI showing live AI intercepts, threat feeds, network correlation graphs, and action buttons (Telecom Block, Bank API Freeze, MHA Dossier Generation).
5. Graph Intelligence View: An interactive force-directed graph visualization that maps scammer infrastructure, victim reports, money mule accounts, and crypto exit nodes.
6. NCRB Report Drafting: Automatic generation of formatted complaints for the National Crime Records Bureau.

Key Design Decisions

- The app has two views toggled from the header:
  - Nodal Portal -- For government/law enforcement officers (dashboard, graph, scanner)
  - Citizen Shield -- A simulated mobile phone UI for citizens (live mic, AI chat)
- The frontend is a single-page React app (App.jsx is the monolithic component file containing everything).
- The backend is a minimal FastAPI server with only 2 POST endpoints -- it acts purely as a bridge between the frontend and the Gemini API.
- The AI model used is Gemini 2.0 Flash (free tier, fast responses).

---

2. TECH STACK

Backend

Component	Technology	Version / Details
Language	Python	3.11
Web Framework	FastAPI	Latest (via pip)
ASGI Server	Uvicorn	Latest (via pip)
Data Validation	Pydantic	Latest (via pip)
AI SDK	google-genai	2.11.0
AI Model (Text)	Gemini 2.0 Flash	gemini-2.0-flash
AI Model (Vision)	Gemini 2.0 Flash	gemini-2.0-flash
Virtual Environment	venv	backend/venv/

Frontend

Component	Technology	Version / Details
Language	JavaScript (JSX)	ES Modules
Framework	React	19.2.7
Build Tool	Vite	8.1.1
CSS Framework	Tailwind CSS	3.4.19
Icons	Lucide React	1.24.0
Graph Visualization	react-force-graph-2d	1.29.1
Linting	ESLint	10.6.0
PostCSS	autoprefixer + tailwindcss	--

Development Tools

Tool	Purpose
VS Code	Primary IDE
Pylance / Pyright	Python type-checking in IDE
Node.js / npm	Frontend package management

---

3. FOLDER STRUCTURE

```
c:\Projects\project-sentinel\
?
??? .vscode\
?   ??? settings.json              # VS Code Python interpreter config
?
??? backend\
?   ??? venv\                      # Python virtual environment (DO NOT COMMIT)
?   ??? __pycache__\               # Python bytecode cache (auto-generated)
?   ??? main.py                    # FastAPI server -- API routes
?   ??? nlp_engine.py              # Gemini AI engine -- ScamClassifier class
?   ??? test_ai.py                 # Quick CLI test script for the AI
?   ??? requirements.txt           # Python dependencies
?
??? frontend\
?   ??? node_modules\              # npm packages (DO NOT COMMIT)
?   ??? public\
?   ?   ??? favicon.svg            # Browser tab icon
?   ?   ??? icons.svg              # SVG icon sprite
?   ??? src\
?   ?   ??? assets\
?   ?   ?   ??? hero.png           # Default Vite hero image (unused)
?   ?   ?   ??? react.svg          # React logo (unused)
?   ?   ?   ??? vite.svg           # Vite logo (unused)
?   ?   ??? App.jsx                # * MAIN APPLICATION FILE (1028 lines)
?   ?   ??? App.css                # Default Vite CSS (mostly unused)
?   ?   ??? index.css              # Tailwind CSS imports
?   ?   ??? main.jsx               # React DOM entry point
?   ??? .gitignore                 # Git ignore rules
?   ??? eslint.config.js           # ESLint configuration
?   ??? index.html                 # HTML entry point
?   ??? package.json               # npm dependencies & scripts
?   ??? package-lock.json          # npm lockfile
?   ??? postcss.config.js          # PostCSS plugins (Tailwind + autoprefixer)
?   ??? tailwind.config.js         # Tailwind CSS configuration
?   ??? vite.config.js             # Vite build configuration
?   ??? README.md                  # Default Vite README
?
??? PROJECT_CONTEXT.md             # * THIS FILE
```

4. DEVELOPMENT ENVIRONMENT SETUP

Prerequisites

- Python 3.11 installed at C:\Users\chinm\AppData\Local\Programs\Python\Python311\
- Node.js (with npm) installed

- Google AI Studio API Key (get one from https://aistudio.google.com/apikey)

Step-by-Step Setup

A. Backend Setup

```
# 1. Navigate to the backend folder
cd c:\Projects\project-sentinel\backend

# 2. Create a Python virtual environment (if not already created)
python -m venv venv

# 3. Activate the virtual environment
.\venv\Scripts\activate

# 4. Install Python dependencies
pip install -r requirements.txt

# 5. (IMPORTANT) Open nlp_engine.py and replace the API key on line 9
#    self.client = genai.Client(api_key="YOUR_API_KEY_HERE")

# 6. Start the backend server
uvicorn main:app --reload
# Server runs at: http://127.0.0.1:8000
```

B. Frontend Setup

```
# 1. Navigate to the frontend folder
cd c:\Projects\project-sentinel\frontend

# 2. Install npm dependencies
npm install

# 3. Start the development server
npm run dev
# Server runs at: http://localhost:5173 (default Vite port)
```

C. Running Both Together

You need two terminal windows/tabs:

- Terminal 1 (Backend): cd backend && .\venv\Scripts\activate && uvicorn main:app --reload
- Terminal 2 (Frontend): cd frontend && npm run dev

Then open http://localhost:5173 in Google Chrome or Microsoft Edge (required for Web Speech API support).

5. BACKEND -- FULL SOURCE CODE & EXPLANATION

5.1 requirements.txt

```
fastapi
uvicorn
pydantic
google-genai
```

These are the only 4 Python packages needed. google-genai is the official Google Generative AI SDK.

5.2 main.py -- FastAPI Server

This is the API server. It exposes 3 routes and imports the AI engine.

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel
from nlp_engine import analyzer  # Importing our AI brain

# Initialize the backend app
app = FastAPI(title="Project Sentinel API")

# Allow our React frontend to talk to this Python backend without security blocks (CORS)
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# Define the exact structure of the data we expect from the frontend
class TranscriptRequest(BaseModel):
    transcript: str
    caller_id: str = "Unknown"

# NEW: Define the structure for incoming document images
class DocumentRequest(BaseModel):
    image_base64: str

# Our basic health check route
@app.get("/")
def read_root():
    return {"status": "Sentinel API Online", "version": "1.0"}

# --- TEXT AI BRIDGE (For the Live Microphone) ---
@app.post("/api/analyze")
def analyze_call(request: TranscriptRequest):
    # 1. Pass the incoming text to our AI engine
    result = analyzer.analyze_transcript(request.transcript)

    # 2. Package the result and send it back to the frontend
    return {
        "success": True,
        "caller_id": request.caller_id,
        "analysis": result
    }

# --- NEW: VISION AI BRIDGE (For the Counterfeit Scanner) ---
@app.post("/api/scan-document")
def scan_document(request: DocumentRequest):
    # Pass the base64 image data to the Gemini Pro Vision engine
    result = analyzer.analyze_document(request.image_base64)

    return {
        "success": True,
        "analysis": result
    }
```

Key Points:

- CORS is fully open (allow_origins=["*"]) -- suitable for development only.
- The analyzer object is a singleton created when nlp_engine.py is imported.
- There are two Pydantic models: TranscriptRequest (text) and DocumentRequest (base64 image).

5.3 nlp_engine.py -- Gemini AI Engine (The Brain)

This is the core AI logic. It contains the ScamClassifier class.

```
import json
import base64
from google import genai

class ScamClassifier:
    def __init__(self):
        # --- INITIALIZE NEW GEMINI SDK ---
        # PASTE YOUR GOOGLE AI STUDIO KEY BELOW
        self.client = genai.Client(api_key="YOUR_API_KEY_HERE")

        # Use the highly-stable, fully free Gemini 2.0 Flash model
        self.fast_model_name = "gemini-2.0-flash"
        self.pro_model_name = "gemini-2.0-flash"

        # 1. Configuration for the FAST MODEL (Live Voice Interception)
        self.fast_config = genai.types.GenerateContentConfig(
            temperature=0.1, # Low temperature makes the AI logical
            response_mime_type="application/json",
            system_instruction="""
            You are 'Project Sentinel', a highly advanced Indian telecom fraud detection AI.
            Analyze the following live phone call transcript. The language may be English, Hindi,...
            Determine if it is a scam call (Digital Arrest, Customs, Money Mule, OTP fraud) or a ...

            Indian Government Guidelines for Detection:
            - CBI, RBI, Police, and Customs NEVER call to arrest people digitally via Skype/Whats...
            - Officials NEVER ask for money to be transferred to "safe accounts" for verification.
            - Scammers use coercion, urgency, and isolation ("Do not tell anyone", "Do not cut th...
            - Relatives asking for money normally is SAFE. Look for psychological manipulation.

            Respond ONLY with this JSON schema:
            {
                "status": "safe" | "warning" | "critical",
                "confidence": "%",
                "details": ""
            }
            """
        )

        # 2. Configuration for the SMART MODEL (Forensic Computer Vision)
        self.pro_config = genai.types.GenerateContentConfig(
            temperature=0.1,
            response_mime_type="application/json"
        )

    def analyze_document(self, base64_image: str) -> dict:
        """
        Takes a base64 encoded image string, decodes it, feeds it to Gemini Pro Vision,
        and extracts signs of forgery from counterfeit Indian currency.
        """
        try:
            prompt = """
            You are a highly advanced Counterfeit Currency Identification Agent for the Reserve B...
            Analyze this image of a suspected Indian Rupee banknote. Your objective is to detect ...

            Perform a deep forensic scan looking specifically for missing or anomalous RBI securi...
            1. Micro-lettering / Microprinting: Look for blurry, illegible, or missing 'RBI', 'Bh...
```

```
            2. Security Thread: Check if the windowed security thread lacks the color-shifting pr...
            3. Intaglio (Raised) Printing: Check for the absence of depth/shadows on the Mahatma ...
            4. Serial Number Panel: Check for misalignments, incorrect spacing, or failure of the...
            5. Bleed Lines & Watermarks: Check for incorrect angular bleed lines on the edges or ...

            Respond ONLY in this exact JSON schema:
            {
                "status": "safe" | "critical",
                "confidence": "%",
                "forgery_markers": ["list", "of", "specific", "fake", "elements", "found", "in", ...
            }
            """

            # The new SDK requires us to decode the base64 string into bytes
            image_bytes = base64.b64decode(base64_image)
            image_part = genai.types.Part.from_bytes(data=image_bytes, mime_type="image/jpeg")

            # Pass the image and prompt to the model
            response = self.client.models.generate_content(
                model=self.pro_model_name,
                contents=[prompt, image_part],
                config=self.pro_config
            )

            result_dict = json.loads(response.text)
            return result_dict

        except Exception as e:
            print(f"Vision AI Error: {e}")
            return {
                "status": "warning",
                "confidence": "0%",
                "forgery_markers": [f"AI Vision analysis failed: {str(e)}"]
            }

    def analyze_transcript(self, text_transcript: str) -> dict:
        """
        Takes a live stream of text, feeds it to Gemini Flash, and returns an instant threat score.
        """
        if not text_transcript or len(text_transcript.strip()) < 10:
            return {"status": "safe", "confidence": "0%", "details": "Audio too short to analyze."}

        try:
            # Send the transcript to the FAST model
            response = self.client.models.generate_content(
                model=self.fast_model_name,
                contents=text_transcript,
                config=self.fast_config
            )

            # Parse the JSON string returned by Gemini into a Python dictionary
            result_dict = json.loads(response.text)
            return result_dict

        except Exception as e:
            print(f"Text AI Error: {e}")
            return {"status": "warning", "confidence": "0%", "details": "AI analysis failed."}
```

```
# Instantiate our engine
analyzer = ScamClassifier()
```

Key Points:

- Both models currently use gemini-2.0-flash (free tier).
- temperature=0.1 ensures deterministic, logical responses.
- response_mime_type="application/json" forces Gemini to output valid JSON.
- The system instruction contains India-specific fraud detection guidelines.
- The analyzer singleton is created at module level (line 116), so it initializes once when the backend starts.

5.4 test_ai.py -- CLI Test Script

```
# test_ai.py
from nlp_engine import analyzer

print("--- PROJECT SENTINEL AI TEST ---\n")

# Test 1: A normal conversation
normal_call = "Hey mom, I am running late from the office. I will transfer the rent money tomorrow."
result1 = analyzer.analyze_transcript(normal_call)
print(f"Normal Call Result: {result1['status']} ({result1['confidence']})")

# Test 2: A Digital Arrest Scam
scam_call = "Hello, this is officer Sharma from CBI. A parcel with your Aadhar has been seized wi...
result2 = analyzer.analyze_transcript(scam_call)
print(f"Scam Call Result: {result2['status']} ({result2['confidence']})")
```

Usage: cd backend && .\venv\Scripts\activate && python test_ai.py

---

6. FRONTEND -- FULL SOURCE CODE & EXPLANATION

6.1 Entry Point Files

index.html

```
    Project Sentinel
```

src/main.jsx

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(


  ,
)
```

src/index.css

```
@tailwind base;
@tailwind components;
@tailwind utilities;
```

6.2 Configuration Files

vite.config.js

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

export default defineConfig({
  plugins: [react()],
})
```

tailwind.config.js

```
/* @type {import('tailwindcss').Config} /
export default {
  content: [
    "./index.html",
    "./src/*/.{js,ts,jsx,tsx}",
  ],
  theme: {
    extend: {},
  },
  plugins: [],
}
```

postcss.config.js

```
export default {
  plugins: {
    tailwindcss: {},
    autoprefixer: {},
  },
}
```

eslint.config.js

```
import js from '@eslint/js'
import globals from 'globals'
import reactHooks from 'eslint-plugin-react-hooks'
import reactRefresh from 'eslint-plugin-react-refresh'
import { defineConfig, globalIgnores } from 'eslint/config'

export default defineConfig([
  globalIgnores(['dist']),
  {
    files: ['*/.{js,jsx}'],
    extends: [
      js.configs.recommended,
      reactHooks.configs.flat.recommended,
      reactRefresh.configs.vite,
    ],
    languageOptions: {
      globals: globals.browser,
      parserOptions: { ecmaFeatures: { jsx: true } },
    },
  },
])
```

6.3 package.json -- Frontend Dependencies

```
{
  "name": "frontend",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "lint": "eslint .",
    "preview": "vite preview"
  },
  "dependencies": {
    "lucide-react": "^1.24.0",
    "react": "^19.2.7",
    "react-dom": "^19.2.7",
    "react-force-graph-2d": "^1.29.1"
  },
  "devDependencies": {
    "@eslint/js": "^10.0.1",
    "@types/react": "^19.2.17",
    "@types/react-dom": "^19.2.3",
    "@vitejs/plugin-react": "^6.0.3",
    "autoprefixer": "^10.5.2",
    "eslint": "^10.6.0",
    "eslint-plugin-react-hooks": "^7.1.1",
    "eslint-plugin-react-refresh": "^0.5.3",
    "globals": "^17.7.0",
    "postcss": "^8.5.16",
    "tailwindcss": "^3.4.19",
    "vite": "^8.1.1"
  }
}
```

6.4 src/App.jsx -- The Main Application (Complete)

This is a monolithic component file (1028 lines). It contains the entire UI including:

- App -- Root component with all state management
- NavItem -- Sidebar navigation button
- GraphIntelligenceView -- Force-directed graph visualization
- CounterfeitScannerView -- Currency image upload + AI scan

Architecture Overview of App.jsx

```
App (Root Component)
??? Header (with Nodal Portal / Citizen Shield toggle)
?
??? IF mainView === 'nodal':
?   ??? Sidebar (NavItem × 3)
?   ?   ??? Live Intercepts (dashboard tab)
?   ?   ??? Graph Intelligence (graph tab)
?   ?   ??? Currency Scanner (counterfeit tab)
?   ?
?   ??? Main Content Area
?       ??? Dashboard Tab
?       ?   ??? Stats Cards (Active Threats, Funds Frozen, Citizen Queries)
?       ?   ??? "Simulate Telecom Feed" Button
?       ?   ??? Live AI Intercepts Feed (threat cards with action buttons)
?       ?   ??? Network Correlation Panel
?       ?
?       ??? Graph Intelligence Tab ->
?       ??? Currency Scanner Tab ->
?
??? IF mainView === 'citizen':
?   ??? Simulated Mobile Phone UI
?       ??? Status Bar (9:41, 5G, battery)
?       ??? App Header (SENTINEL logo + 12 language selector)
?       ??? IF citizenTab === 'shield':
?       ?   ??? Shield Status Circle (safe/warning/critical)
?       ?   ??? Live Audio Transcript Display
?       ?   ??? Mic Toggle Button
?       ??? IF citizenTab === 'assistant':
?       ?   ??? Chat Messages List
?       ?   ??? Chat Input Form
?       ??? Bottom Navigation (Call Shield | AI Assistant)
?       ??? NCRB Report Modal (overlay)
?
??? Confirmation Modal (for Nodal actions)
??? Toast Notification
```

State Variables

Variable	Type	Purpose
mainView	string	'nodal' or 'citizen' -- top-leve...
activeTab	string	'dashboard', 'graph', or 'counte...
activeThreats	array	List of threat objects in the li...
intercepted	array	List of threat IDs that have bee...
filter	string	'all' or 'critical' -- threat fe...
confirmDialog	object	Currently open confirmation dialog
toast	string	Toast notification text (null = ...
isAnalyzing	boolean	Loading state for "Simulate Tele...
citizenTab	string	'shield' or 'assistant' -- citiz...
callLanguage	string	BCP 47 language code for speech ...
isListening	boolean	Whether the microphone is active
liveTranscript	string	Real-time speech-to-text output
shieldStatus	string	'safe', 'warning', or 'critical'
chatInput	string	Current text in the chat input f...
isChatLoading	boolean	Loading state for the AI assista...
showNcrbModal	boolean	Whether the NCRB report modal is...
chatMessages	array	Array of chat message objects

Key Functions

Function	Purpose
toggleListen()	Starts/stops the browser's Speec...
evaluateWithGemini()	Sends transcript to backend /api...
handleChatSubmit()	Sends chat message to backend, r...
simulateLiveCall()	Sends a hardcoded scam transcrip...
requestAction()	Opens the confirmation dialog fo...
executeAction()	Marks a threat as "actioned" aft...

Supported Languages (Speech Recognition)

The Citizen Shield supports 13 Indian languages:

Code	Language
en-IN	English
hi-IN	Hindi (?????)
mr-IN	Marathi (?????)
bn-IN	Bengali (?????)
te-IN	Telugu (??????)
ta-IN	Tamil (?????)
gu-IN	Gujarati (???????)
ur-IN	Urdu (????)
kn-IN	Kannada (?????)
or-IN	Odia (?????)
ml-IN	Malayalam (??????)
pa-IN	Punjabi (??????)
as-IN	Assamese (???????)

Sub-Components

NavItem (line 708-714): Simple sidebar navigation button. Accepts icon, label, active, onClick.

GraphIntelligenceView (line 716-888):

- Uses react-force-graph-2d library for interactive graph visualization.
- Hardcoded demo data with 8 nodes (Scammer, IPs, Victims, Mule accounts, Crypto wallet) and 8 links.
- Clicking a node shows a forensics panel with details.
- "Export Court-Admissible Package" button generates and downloads a text file dossier.

CounterfeitScannerView (line 891-1028):

- Image upload with drag-and-drop zone.
- Converts uploaded file to base64 using FileReader API.
- Sends base64 data to backend /api/scan-document endpoint.
- Displays scan results: status (safe/critical), confidence %, and list of forgery markers.

---

7. API CONTRACT (Backend <-> Frontend)

Base URL: http://127.0.0.1:8000

GET /

Purpose: Health check.

Response:

```
{
  "status": "Sentinel API Online",
  "version": "1.0"
}
```

POST /api/analyze

Purpose: Analyze text transcript for scam indicators.

Request Body:

```
{
  "transcript": "Hello, this is officer Sharma from CBI...",
  "caller_id": "Live Citizen Mic"    // optional, defaults to "Unknown"
}
```

Response (Success):

```
{
  "success": true,
  "caller_id": "Live Citizen Mic",
  "analysis": {
    "status": "critical",           // "safe" | "warning" | "critical"
    "confidence": "98%",
    "details": "Impersonation of CBI officer with digital arrest coercion tactics detected."
  }
}
```

Used by:

- Live Mic Shield (evaluateWithGemini())
- AI Fraud Assistant Chat (handleChatSubmit())
- Nodal Officer "Simulate Telecom Feed" (simulateLiveCall())

POST /api/scan-document

Purpose: Analyze an image of Indian currency for counterfeit detection.

Request Body:

```
{
  "image_base64": "/9j/4AAQSkZJRg..."   // base64 encoded JPEG/PNG
}
```

Response (Success):

```
{
  "success": true,
  "analysis": {
    "status": "critical",            // "safe" | "critical"
    "confidence": "87%",
    "forgery_markers": [
      "Micro-lettering between Gandhi portrait and vertical band is blurry and illegible",
      "Security thread lacks proper color-shifting property",
      "Serial number panel shows inconsistent spacing"
    ]
  }
}
```

Used by:

- Counterfeit Scanner View (executeScan())

---

8. HOW THE AI ENGINE WORKS

Text Analysis Flow

```
User speaks into mic (or types in chat)
       ?
Browser Web Speech API converts speech to text
       ?
Frontend sends text via POST to /api/analyze
       ?
FastAPI receives TranscriptRequest
       ?
analyzer.analyze_transcript() is called
       ?
Text is sent to Gemini 2.0 Flash with system instruction
(temperature=0.1, response_mime_type="application/json")
       ?
Gemini returns JSON: {status, confidence, details}
       ?
FastAPI wraps it in {success, caller_id, analysis}
       ?
Frontend updates UI based on "status" value:
  - "safe"     -> Green shield, no alert
  - "warning"  -> Yellow warning state
  - "critical" -> Red pulsing alert, disconnect warning
```

Vision Analysis Flow

```
User uploads image of banknote
       ?
FileReader converts image to base64 string
       ?
Frontend sends base64 via POST to /api/scan-document
       ?
FastAPI receives DocumentRequest
       ?
analyzer.analyze_document() is called
       ?
base64 is decoded to bytes -> genai.types.Part.from_bytes()
       ?
Image + detailed forensic prompt sent to Gemini 2.0 Flash
(temperature=0.1, response_mime_type="application/json")
       ?
Gemini returns JSON: {status, confidence, forgery_markers}
       ?
FastAPI wraps it in {success, analysis}
       ?
Frontend displays verdict + list of forgery markers
```

AI System Prompt (Text Analysis)

The AI is instructed to act as an Indian telecom fraud detection agent with these rules:

- CBI, RBI, Police, and Customs NEVER call to arrest people digitally.
- Officials NEVER ask for money to "safe accounts."
- It looks for: coercion, urgency, isolation tactics.
- Normal family conversations about money are classified as safe.
- Must respond in strict JSON format with status, confidence, and details.

AI System Prompt (Vision Analysis)

The AI acts as an RBI Counterfeit Currency Agent checking 5 specific security features:

1. Micro-lettering / Microprinting
2. Security Thread (color-shifting)
3. Intaglio (Raised) Printing
4. Serial Number Panel (alignment, spacing)
5. Bleed Lines & Watermarks

---

9. VS CODE CONFIGURATION

.vscode/settings.json

```
{
    "python.defaultInterpreterPath": "C:\\Projects\\project-sentinel\\backend\\venv\\Scripts\\pyt...
    "python.analysis.pythonPath": "c:\\Projects\\project-sentinel\\backend\\venv\\Scripts\\python...
    "python.analysis.extraPaths": [
        "./backend"
    ]
}
```

Why this matters:

- python.defaultInterpreterPath tells VS Code which Python to use for running code.
- python.analysis.pythonPath tells Pylance/Pyright where to find installed packages (fixes "Cannot find module" errors).
- python.analysis.extraPaths adds the backend folder to the import search path so Pylance can resolve from nlp_engine import analyzer.

---

10. KNOWN ISSUES & TROUBLESHOOTING

Issue 1: "Cannot find module google.genai"

Cause: IDE (Pylance) is using the global Python interpreter instead of the virtual environment.

Fix: In VS Code, press Ctrl+Shift+P -> "Python: Select Interpreter" -> choose backend\venv\Scripts\python.exe.

Issue 2: "Could not resolve interpreter path"

Cause: VS Code settings point to a venv that hasn't been created yet or the path is wrong.

Fix: Ensure the venv exists at backend/venv/ and the path in .vscode/settings.json matches.

Issue 3: Speech Recognition not working

Cause: Web Speech API is only supported in Chrome and Edge. Firefox/Safari do not support it.

Fix: Use Google Chrome or Microsoft Edge.

Issue 4: "Cannot connect to Python AI Backend"

Cause: The FastAPI backend is not running.

Fix: Start the backend with cd backend && .\venv\Scripts\activate && uvicorn main:app --reload.

Issue 5: CORS errors in browser console

Cause: Frontend and backend are on different ports (5173 vs 8000).

Fix: This is already handled -- the backend has allow_origins=["*"] in CORS middleware.

Issue 6: Gemini API returns errors

Cause: Invalid or expired API key, or quota exceeded.

Fix: Get a fresh API key from https://aistudio.google.com/apikey and update nlp_engine.py line 9.

11. FUTURE SCOPE / ROADMAP

The following features are not yet implemented but are part of the project vision:

1. File Upload in Chat: The "UploadCloud" button in the AI Assistant chat currently shows a toast "not implemented yet."
2. Real Telecom Integration: Replace the "Simulate Telecom Feed" button with actual TRAI/Telecom API feeds.
3. Database: No database is used -- all data is in-memory. Adding PostgreSQL or MongoDB would enable persistent threat logging.
4. Authentication: No login system. In production, Nodal Officers would need JWT-based auth with role-based access.
5. Real NCRB API Integration: The NCRB report drafting currently only generates a visual mock. Real integration with cybercrime.gov.in would be needed.
6. Production Deployment: The app runs on localhost. For deployment, consider:
   - Backend: Deploy on Google Cloud Run or Railway
   - Frontend: Deploy on Vercel or Netlify
   - Use environment variables for API keys (not hardcoded)

END OF DOCUMENT

This document contains the complete context of the Project Sentinel codebase as of July 12, 2026. Any AI assistant or developer reading this should have full understanding to immediately set up, run, debug, and extend this project.

END OF DOCUMENT

This document contains the complete context of the Project Sentinel codebase. Any AI assistant or developer reading this should have full understanding to immediately set up, run, debug, and extend this project.